



SpeedGameManager: Менеджер Скорости Игры

SpeedGameManager.

В Unity скорость игры можно контролировать с помощью `Time.timeScale`. Однако такое решение имеет ряд ограничений и недостатков. Например могут возникнуть проблемы с анимацией UI, воспроизведением звуков и прочие, которые потребуют поиска решений, обходов и “костылей”.

Поэтому в разработке чаще предпочитается логическая пауза через bool систему событий, или же подобное моему решению ниже, только проще.

В моём случае (как и во всех TowerDefence) мне потребовалось создать систему множителей скорости (x0, x1, x2, x4).

Скрипт, контролирующий скорость игры:

```
1 using System;
2 using UnityEngine;
3
4 public enum GameSpeed
5 {
6     Pause, X1, X2, X4
7 }
8
9 public class SpeedGameManager : Manager<SpeedGameManager>
10 {
11     public float SpeedMultiplier { get; private set; } = 0f;
12     public GameSpeed CurrentSpeed { get; private set; } = GameSpeed.Pau
13     public GameSpeed PastSpeed = GameSpeed.Pause;
14
15     protected override void Awake()
16     {
17         base.Awake();
18     }
19
20     private void SetSpeed(GameSpeed speed, float multiplier)
21     {
22         PastSpeed = CurrentSpeed;
23         CurrentSpeed = speed;
24         SpeedMultiplier = Mathf.Max(0f, multiplier);
25
26         OnSpeedGameChanged?.Invoke(CurrentSpeed);
27     }
28
29     public void Pause() => SetSpeed(GameSpeed.Pause, 0f);
30     public void Resume() => SetSpeed(GameSpeed.X1, 1f);
31     public void Speed2x() => SetSpeed(GameSpeed.X2, 2f);
32     public void Speed4x() => SetSpeed(GameSpeed.X4, 4f);
33     public void SetSpeedState(GameSpeed speed)
34     {
35         switch (speed)
36         {
37             case GameSpeed.Pause:
38                 Pause();
39                 break;
```

```

40         case GameSpeed.X1:
41             Resume();
42             break;
43         case GameSpeed.X2:
44             Speed2x();
45             break;
46         case GameSpeed.X4:
47             Speed4x();
48             break;
49         default:
50             Pause();
51             break;
52     };
53 }
54
55 public void SetPastSpeed()
56 {
57     SetSpeedState(PastSpeed);
58 }
59
60 public event Action<GameSpeed> OnSpeedGameChanged;
61 }
62 // SpeedGameManager.cs

```

Конечно же - никаких TimeScale. Менеджер будет работать через простое изменение поля SpeedMultiplier и уведомление всех подписчиков события, которые хотят реагировать на скорость.

Изначально было решено сделать интерфейс ISpeedAffectable, который бы напоминал, что для определенного класса необходимо реализовать SetSpeedMultiplier, этот метод бы слушал событие OnSpeedGameChanged, и менял кэшированное поле скорости игры на новое.

Был также вариант реализовать логику через абстрактный класс, который бы делал автоматически подписку и отписку на это событие, чтобы не дублировать код. Однако наследование класса могло бы оказать неприятности при дальнейшей разработке, когда классу (например юнита) понадобилось бы наследовать ещё что-то. Пришлось бы заморочиться над композицией.

Пример того, как бы происходило вычисление чего-либо с учётом скорости игры:

```

1 private void MoveTowardsTarget()
2 {
3     Vector3 direction = (targetPoint - transform.position).normalized;
4     UpdateAnimations(direction);
5
6     transform.position = Vector3.MoveTowards(transform.position,
7         targetPoint, moveSpeed * Time.deltaTime * speedGame);
8
9     if (Vector3.Distance(transform.position, targetPoint) < 0.05f)
10    {
11        currentZoneIndex++;
12        if (currentZoneIndex >= pathZones.Length)
13        {
14            StopMovement();
15            return;
16        }
17    }
18 }

```

```

17     SetNextTarget();
18 }
19 }
20 // EnemyMovement.cs - Метод движения врага.

```

Здесь нас интересует строка вычисления позиции, где в MoveTowards мы добавляем параметр speedGame, который было задумано кешировать и держать внутри классов.

Однако, данная махинация с постоянной подпиской / отпиской множества объектов показалась избыточной и более затратной, чем прямой доступ к SpeedGameManager.Instance.SpeedMultiplier свойству. Для простых операций, по типу вычисления движения и прочего, что требует простого получения поля из синглтона, было решено отказаться от событийности и использовать поле напрямую. Соответственно: можно убрать контракт с интерфейсом.

Однако сам интерфейс и событийность всё ещё понадобится, когда мы дойдём до звуков, которые нужно будет менять на ходу (обновлять по событию) или в какой-нибудь более сложной логике, которая должна запускаться при смене скорости игры.

Upd: Далеко ходить не пришлось. EnemyAnimatorController:

```

1 public void PlayWalk(EnemyDirection direction, float speedMultiplier =
2 {
3     rightFlip = false;
4
5     switch (direction)
6     {
7         case EnemyDirection.Up: currentAnimation = EnemyAnimation.Walk_
8         case EnemyDirection.Down: currentAnimation = EnemyAnimation.Wal
9         case EnemyDirection.Left: currentAnimation = EnemyAnimation.Wal
10        case EnemyDirection.Right: currentAnimation = EnemyAnimation.Wa
11            rightFlip = true; break;
12        default: currentAnimation = EnemyAnimation.Walk_Side; break;
13    }
14
15    PlayAnimation();
16    animator.speed = speedMultiplier * SpeedGameManager.Instance.SpeedM
17 }
18 // EnemyAnimationController.cs

```

В конце метода аниматор меняет скорость на предложенную извне. Сделано это для того, чтобы в ходе усиления врагов, когда их скорость станет выше, аниматор мог слегка ускориться, и ощущение ускорения было более выраженным анимацией в том числе. Метод вызова анимации смерти сбрасывал скорость на 1f. Однако с введением менеджера скорости игры они оба в добавок умножаются на глобальную скорость: SpeedGameManager.Instance.SpeedMultiplier;

✖ Вот только этого не достаточно, ведь уже проигрывающаяся анимация не поменяет свою скорость, пока метод не вызовет извне.

Для этого нам поможет интерфейс, и событийность. Чтобы при изменении скорости наш контролер сказал аниматору, что пора ускорять уже ТЕКУЩУЮ анимацию:

```
1 private void OnEnable()
2 {
3     SpeedGameManager.Instance.OnSpeedGameChanged += SetAnimatorSpeed;
4 }
5
6 private void OnDisable()
7 {
8     SpeedGameManager.Instance.OnSpeedGameChanged -= SetAnimatorSpeed;
9 }
10
11 private void SetAnimatorSpeed(float newSpeed)
12 {
13     animator.speed = newSpeed;
14 }
15 // EnemyAnimationController.cs
```

Мы подписываемся на событие обновления скорости, не забываем отписаться при выключении объекта. Реализуем сам метод, меняя скорость аниматора. В таком случае скорость изменится сразу, как событие сработает.

Интерфейс в данном случае не был применен. И причина странная: я просто решил назвать метод более читаемо для этого класса. Возможно стоило бы указать на контракт с интерфейсом и создать какой-то обобщенный по названию метод, внутри которого я бы вызвал SetAnimatorSpeed. Это бы просто позволяло указать, что данный скрипт реагирует на время игры.

SpeedGameController.

Для того, чтобы игрок мог контролировать время - нужен специальный интерфейс блок.



```
1 using UnityEngine;
2 using UnityEngine.InputSystem;
3 using UnityEngine.InputSystem.HID;
4 using UnityEngine.UI;
5
6 public class SpeedController : MonoBehaviour
7 {
8     public Button pauseButton;
9     public Button normalButton;
10    public Button _2xButton;
11    public Button _4xButton;
12
13    private Button ActiveButton;
14}
```

```
15 private Color defaultColor;
16 private Color inactiveColor = new Color(255, 255, 255, 0.3f);
17
18 private void Start()
19 {
20     SpeedGameManager.Instance.OnSpeedGameChanged += OnSpeedChanged
21     defaultColor = pauseButton.image.color;
22     ActiveButton = pauseButton;
23     ButtonOff();
24 }
25
26 private void OnDestroy()
27 {
28     SpeedGameManager.Instance.OnSpeedGameChanged -= OnSpeedChanged
29 }
30
31 private void ButtonOff()
32 {
33     pauseButton.image.color = inactiveColor;
34     normalButton.image.color = inactiveColor;
35     _2xButton.image.color = inactiveColor;
36     _4xButton.image.color = inactiveColor;
37     ActiveButton.image.color = defaultColor;
38 }
39
40 public void OnPause()
41 {
42     if (GameFlowManager.Instance.State != GameState.Playing)
43         return;
44
45     SpeedGameManager.Instance.Pause();
46 }
47 public void OnNormal()
48 {
49     if (GameFlowManager.Instance.State != GameState.Playing)
50         return;
51
52     SpeedGameManager.Instance.Resume();
53 }
54
55 public void On2x()
56 {
57     if (GameFlowManager.Instance.State != GameState.Playing)
58         return;
59
60     SpeedGameManager.Instance.Speed2x();
61 }
62
63 public void On4x()
64 {
65     if (GameFlowManager.Instance.State != GameState.Playing)
66         return;
67
68     SpeedGameManager.Instance.Speed4x();
69 }
70
71 public void KeyPause(InputAction.CallbackContext ctx)
72 {
73     if (!ctx.performed) return;
74     OnPause();
75 }
76
77 public void KeyNormal(InputAction.CallbackContext ctx)
78 {
79     if (!ctx.performed) return;
80     OnNormal();
81 }
```

```

82
83     public void Key2x(InputAction.CallbackContext ctx)
84     {
85         if(ctx.performed) return;
86         On2x();
87     }
88
89     public void Key4x(InputAction.CallbackContext ctx)
90     {
91         if (!ctx.performed) return;
92         On4x();
93     }
94
95     private void OnSpeedChanged(GameSpeed gameSpeed)
96     {
97         ActiveButton = gameSpeed switch
98         {
99             GameSpeed.Pause => pauseButton,
100             GameSpeed.X1 => normalButton,
101             GameSpeed.X2 => _2xButton,
102             GameSpeed.X4 => _4xButton,
103             _ => ActiveButton
104         };
105
106         ButtonOff();
107     }
108 }
109 // SpeedController.cs

```

Данный контроллер в верхней правой части экрана представляет из себя 4 кнопки: пауза, обычная скорость, 2x, 4x. Первые версии этого скрипта были перегружены и стягивали ответственность на себя. Позже всё было упорядочено, теперь скрипт контроллера отвечает только за запрос к SpeedGameManager, не выполняя никаких функций, за исключением самого UI.

```

1 private void ButtonOff()
2 {
3     pauseButton.image.color = inactiveColor;
4     normalButton.image.color = inactiveColor;
5     _2xButton.image.color = inactiveColor;
6     _4xButton.image.color = inactiveColor;
7     ActiveButton.image.color = defaultColor;
8 }

```

Метод кажется каким-то абсурдно простым и странным. Я нашёл такое решение заманчивым 😊. Всем кнопкам присваивается затемнение, словно она неактивная, а после активная выключает затемнение. Активная кнопка ранее становилась таковой прямо в вызываемом методе кнопки (метод OnPause внутри присваивал pauseButton в ActiveButton). Сейчас логика немного изменилась.

Странный маршрут:

сначала мы нажимаем кнопку →

затем обращаемся в менеджер →

тот меняет скорость на нужную и вызывает событие →

мы возвращаемся в контроллер и видим, что подписаны на это событие > нашей реакцией будет как-раз установка активной кнопки в зависимости от пришедшего типа `enum` (состоящего из четырех значений), и сброс кнопок.

Путь кажется странным, потому что мы делаем круг. Вызываем метод у менеджера, чтобы тот вызвал метод у нас же снова. Почему бы не обойтись без событий и подписок, просто внутри (как я упомянул выше) вызывать `ButtonOff()`?

Причина проста: менеджером могут управлять не только кнопки, но и различные ситуации (другие менеджеры): ESC-пауза, экран смерти, победы, выход в меню, старт игры, кнопка пре-старта. Они могут установить паузу, а кнопки об этом знать не будут и они просто не сбросятся, активная кнопка не подсветится.

Разжую ещё один момент для тех, кто зачем-то залез в эту документацию и быть может спросит: почему бы просто тогда ESC-паузе не прожимать кнопку самой? - Потому, что `SpeedController` не является менеджером. Зачем лезть в контроллер, более предназначенный для взаимодействий с игроком, когда есть оригинальный менеджер (без прослоек). + Пришлось бы везде указывать на него ссылки, ведь он не является синглтоном, или делать его методы статическими.

Коротко: контроллер - для внешнего влияния, менеджер - для внутреннего.

напрямую - через контроллер, методы - обращаются через главный менеджер, и он же уведомит контроллер.

Управление с клавиатуры.

Для удобства игроков я также добавил клавиши быстрого доступа: 1,2,3,4 - для смены скоростей игры. Из интересного: разобрался с новой системой отслеживания клавиш от Unity6, которая из коробки работает через события (калбеки), в версии 2022 это делалось иначе. Впрочем, предоставляемая технология из коробки работает лучше.

```
1 public void KeyPause(InputAction.CallbackContext ctx)
2     {
3         if (!ctx.performed) return;
4         OnPause();
5     }
```

Поэтому методы переключения скорости дублируются. Вернее у них есть формат для принятия `CallbackContext`'ов от `InputAction` системы. Сделал ультра простую карту событий. Добавил компонент на контроллер, указал ссылки на методы. Всё прекрасно работает и переключается.

А в методах мы просто проверяем отработало ли событие нажатия клавиши, и запускаем необходимый метод.

Карта событий:

